

Benchmark guide: what's out there, how each is scored, and what we use

Faisal · July 16, 2026 (rev. 2, after the adversarial verification pass) · companion to the model decision memo. Every date and scoring claim below was verified against the paper, repo, or dataset page this week; the plan-critical claims were additionally re-verified by reading the actual code.

The key concept: program-scored vs. judge-scored

Every benchmark needs a way to decide how good an output is. There are only two ways to do it, and the difference drives every decision in this document.

Program-scored

A piece of **code** computes the score. It checks facts about the file: did the generated code run? Does the slide contain the required text? Do two boxes overlap? Is the color the exact hex the theme demands?

Same input → same score, every single time.
Costs nothing to run. Cannot be sweet-talked.

Judge-scored

Another **AI** looks at the output and gives an opinion — a rubric grade, a 0–5, a "which of these two is better." (Also called "LLM-as-judge" or "VLM-as-judge" when the judge looks at images.)

Scores wobble between runs. Costs an API call every time. Can be impressed, and therefore fooled.

Why this matters so much for us — three reasons:

1. **Training cost.** GRPO works by scoring thousands of attempts and nudging the model toward the good ones. A program scores each attempt for free in milliseconds. A judge charges frontier-API prices per attempt — the reward bill can exceed the training bill.
2. **Reward hacking.** A model being trained will learn to exploit whatever scores it. You cannot fool a program that greps the XML for a missing footer. You *can* learn to impress a judge — flashier layouts, confident wording — without actually getting better. The trained model looks improved and isn't.

3. **Credibility.** "+3% on a program-computed metric" is a fact anyone can re-run. "+3% in the opinion of GPT-5" is an argument. Our entire pitch rests on this distinction — Michael's grader is deterministic OOXML checks precisely so no one can dispute the baseline.

One subtlety: some benchmarks are **hybrid**, and some judge-scored ones dress up as programmatic. PPTArena feeds "structural diffs" to its judges — but the program only *prepares evidence*; the AI still *decides the grade*. That's judge-scored. The test is: who produces the final number, code or a model?

The three filters for a trainable benchmark

- 1. **Program-scored** — otherwise the reward is expensive and hackable (above).
- 2. **Training tasks exist** — GRPO needs tasks to practice on; an eval-only set of 120–800 tasks gives nothing to train against.
- 3. **Contamination-safe measurement.** Qwen3.6-27B was released April 22, 2026, and Alibaba publishes *no data cutoff and no decontamination statement* (we checked the card, blog, GitHub, and all 47 HF discussions). So any benchmark published much before April 2026 may be in its training data. Crucial nuance: **contamination only poisons the measurement, never the training** — training on data the model has seen is what fine-tuning is. The *held-out eval* is what must be clean.

The full landscape (verified July 2026)

BENCHMARK	DATE	TASK	SCORING	TRAIN DATA	CLEAN FOR 27B?	OUR USE
SlidesBench (AutoPresent)	Jan 2025	Make one pptx slide from an instruction (code-gen)	Program (execution + text/color/position match)	7,000 tasks	No — 15 mo. exposure	Training proof-of-life (eval on perturbed set)
AeSlides	Apr 21, 2026	Generate HTML slides; layout quality	Program only (aspect, whitespace, collisions, balance)	5,704 tasks — NOT released ("proprietary constraints")	Yes — provably	Methods reference only — code + train set withheld; recipe ideas transfer
PPT-Eval (Microsoft, ICML)	Jun 30, 2026	Edit/create in real PowerPoint (GUI + API agents)	Hybrid — 59/120 tasks fully deterministic, rest Claude-judged	None (eval-only, by design)	Yes	Clean pptx eval checkpoint (the 59 deterministic tasks)

PPTArena (editing; Ofengenden)	Dec 2025	800+ in-place edits on 100 real decks, incl. master styles	Judge (GPT-5 + Gemini, dual VLM)	None	Mostly (4 mo. window)	External comparison eval for 27B vs 480B; cite in report
PresentBench	Mar 2026	Deck from background materials (238 instances)	Judge (Gemini, ~54 checklist items each)	None	Borderline	Read for failure modes; cite headroom (best system 62.5)
UniPPTBench	May 17, 2026	Deck from vague prompt / long doc / multi-source (126 tasks)	Judge (Gemini 3 Flash)	None; data not yet released	Yes	Not usable yet — watch
X+Slides	Jun 17, 2026	Audience- conditioned decks (8,133 probes)	Judge + formulaic aggregation	None	Yes	Not used
DECKBench	Feb 2026	Paper→HTML deck + 5-turn editing (294 pairs)	Mixed (CLIP/embedding + judge)	None	Probably	Not used (HTML, non-commercial license)
SlideRL / Learning- to-Present	Mar 2026	RL env, HTML slides via tool use	Mixed (structural checks + judge aesthetics)	288 trajectories	Borderline	Reference for env design
SlidesGen-Bench	Jan 2026	Slide quality metrics (content/aesthetic s/editability)	Program (computational metrics)	Human-pref data	Borderline (3 mo.)	Possible extra reward components
SWE-rebench (off- domain)	monthly → Mar 2026	Fix real GitHub issues	Program (tests pass/fail)	21k tasks (stale: ≤Apr 2025)	2026 eval splits: yes	Off-domain fallback only
powerbench (ours)	Jul 2026	Template + source text → conforming deck	Program (8 OOXML checks) + separate LLM content check	Task generator (ours)	Yes — private	The real benchmark — everything above is rehearsal

What we're doing with each — the plan

- **Train on SlidesBench, measure on a perturbed set — confirmed viable at code level, with three requirements.** A verification pass (July 16) read the actual eval code: the reference-based metrics (text similarity, color via CIEDE2000, position, element matching in `evaluate/page_eval.py`) are deterministic pptx-vs-pptx comparisons with no judge — so "perturb the reference, re-score" works directly. Requirements: (1) *use only that reference-based family* — the reference-free eval is GPT-4o-judged and one metric uses CLIP; exclude both; (2) *budget for self-fixing a dead repo* — no commits since May 2025, 8 open issues including a README-following crash nobody answered; (3) *source training tasks ourselves* — the official 7k download path is broken (users trade unofficial 31 GB mirrors). Cleanest fix: mint our own

training tasks with the repo's `create_dataset.py` from decks we choose — which also fixes the fact that the official test pool comes from just 10 decks.

- **AeSlides is demoted to a methods reference — it cannot be run.** Despite the paper claiming a public release, the repo explicitly withholds the full system ("proprietary constraints"): no train set (the only HF dataset is a 2,056-row preference bench), no reward pipeline, no training configs; the recipe ran on slime (not verl) with a 30B-MoE (3B active — not a dense-27B analog); and its headline programmatic gains are circular (eval metrics = training rewards; the paper documents patching reward hacking). What still transfers: the +7.6% *human-eval* gain de-risks our +2–3% target directionally, verl natively supports its RL tricks (clip-higher, token-level loss, GSPO, KL), and its reward-normalization snippets are reusable.
- **PPTArena (editing) is the head-to-head eval** for 27B vs 480B — closest public task to ours, contamination-clean for the 480B. Judge-scored, so it is never a training reward; judge bills go on the OpenAI hackathon credits.
- **PPT-Eval's 59 deterministic tasks** are a free, provably-clean pptx eval checkpoint for both models. **Not yet verified hands-on:** whether they run standalone against an API agent (vs. their GUI sandbox) — one hour with the repo settles it; same for PPTArena's per-task judge cost. Both checks are scheduled before we spend on either.
- **powerbench is the benchmark that matters.** Private (zero contamination by construction), program-scored, in-domain. The July 24 k=8 baseline is the number the pitch stands on.

Naming trap for the team: "PPTArena" is two different things. (1) The Dec 2025 deck-*editing* benchmark by Ofengenden et al. (VLM-judged, 100 decks) — the one in our table. (2) Microsoft's computer-use benchmark, which was *renamed to PPT-Eval* for ICML 2026 (leftover "PPTArena" strings are still in their camera-ready). Don't conflate them in the report.

Ammunition for the Anthropic pitch

- **The gap we fill is documented, not claimed.** Verified negative from the sweep: as of July 2026, *no pptx-native RL environment with programmatic rewards exists* — all published slide-RL work (AeSlides, SlideRL) is HTML-based. Powerbench is first.
- **Microsoft says the quiet part out loud.** From PPT-Eval's own limitations: hand-built rubrics require so much human editing it is "*preventing us from scaling arbitrarily, which would be useful in agent training scenarios (e.g., reinforcement learning).*" Our deterministic template-

conformance grader is exactly the thing they say can't be scaled — because they graded aesthetics with judges, and we grade conformance with code.

- **The domain is externally confirmed unsaturated:** Claude Sonnet 4 $\approx$ 43% success on PPTArena editing; Claude 4.5 Opus 45% success on PPT-Eval (humans: 80%); best system on PresentBench scores 62.5/100. Deck editing is hard for everyone — it's not a quirk of our grader.

Sources (all fetched/verified July 15–16, 2026): AutoPresent/SlidesBench arXiv 2501.00912 + repo · AeSlides arXiv 2604.22840 + github.com/ympan0508/aeslides · PPT-Eval arXiv 2606.31154 + github.com/microsoft/PPTEval (ICML 2026) · PPTArena (editing) arXiv 2512.03042 + github.com/michaelofengend/PPTArena · PresentBench arXiv 2603.07244 + HF dataset · UniPPTBench arXiv 2605.17356 · X+Slides arXiv 2606.19256 + github.com/OpenDataBox/X-SlidesBench · DECKBench arXiv 2602.13318 · SlideRL arXiv 2603.16839 · SlidesGen-Bench arXiv 2601.09487 · SWE-rebench (nebius, HF) · Terminal-Bench 2.1/Harbor · Qwen3.6-27B card/blog/GitHub (no cutoff or decontamination statement found).
Research: two multi-agent verification sweeps, 30+ sources fetched directly.